PROJET DOM - POKÉMON BATTLE

OBJECTIF DU PROJET

Créer un jeu de combat Pokémon en HTML, CSS et JavaScript où deux Pokémon s'affrontent tour par tour jusqu'à ce que l'un d'eux soit K.O.

PAGE D'ACCUEIL

Éléments requis :

- **Titre du jeu** : Nom créatif pour votre jeu (ex: "Poké Fight", "Battle Arena", etc.)
- **Animation d'entrée** : Une Pokéball qui tombe et tremble par exemple
- **Bouton "Lancer le jeu"** : Qui redirige vers index.html
- **Design au choix** : Couleurs, animations, effets sont libres

Critères d'évaluation :

- Design attractif et cohérent avec l'univers Pokémon
- Animation fluide
- Navigation fonctionnelle vers le jeu

JEU DE COMBAT

A. SÉLECTION DES POKÉMON

Consignes :

1. **Choisir 3 Pokémon** différents (PAS obligatoirement Dracaufeu, Tortank et Florizarre)
    a. Vous pouvez choisir n'importe quels Pokémon
    b. Trouvez leurs sprites sur : https://www.pokepedia.fr ou https://pokemondb.net
2. **Pour chaque Pokémon, définir :**
    a. Nom

    b. Image/Sprite (lien URL)
    c. Points de vie (HP) entre 300 et 400
    d. 4 attaques différentes
3. **Pour chaque attaque, indiquer :**
    a. Nom de l'attaque
    b. Type (feu, eau, plante, électrique, normal, etc.)
    c. Puissance (entre 70 et 120)
    d. Précision (entre 0.80 et 1.00)

Exemple de structure :

```
let pkmList = [
    ['Pikachu', 'URL_DU_SPRITE', 320, [
        ['Tonnerre', 'électrique', 90, 0.90],
        ['Vive-attaque', 'normal', 75, 0.95],
        ['Cage-éclair', 'électrique', 85, 0.85],
        ['Queue de fer', 'steel', 80, 0.90]
    ]],
    // ... 2 autres Pokémon
];
```

Écran de sélection :

- **Au début du jeu** : Afficher les 3 Pokémon avec leurs sprites
- **L'utilisateur clique** sur un Pokémon pour le choisir
- **Après sélection** : L'écran disparaît et le combat commence
- **L'adversaire** : Spawn aléatoirement parmi les 3 Pokémon

B. SYSTÈME DE TYPES

Vous devez créer un système de forces/faiblesses pour vos Pokémon.

Exemples de relations de types :

- Feu > Plante (super efficace)
- Eau > Feu (super efficace)
- Plante > Eau (super efficace)
- Électrique > Eau (super efficace)
- Sol > Électrique (super efficace)

C. INTERFACE DE COMBAT

Éléments OBLIGATOIRES à afficher :

1. **Sprite du Pokémon joueur** (en bas à gauche)
    a. Taille : environ 280x280px
    b. Position : bas gauche de l'écran
2. **Sprite du Pokémon ennemi** (en haut à droite)
    a. Taille : environ 200x200px
    b. Position : haut droit de l'écran
3. **Informations du Pokémon joueur** (en haut à gauche)
    a. Nom du Pokémon
    b. HP actuels / HP maximum
    c. Barre de vie visuelle (vert → orange → rouge)
4. **Informations du Pokémon ennemi** (en haut à droite)
    a. Nom du Pokémon
    b. HP actuels / HP maximum
    c. Barre de vie visuelle (vert → orange → rouge)
5. **Zone de texte centrale** (en bas, au centre)
    a. Messages de combat
    b. Annonces d'attaques
    c. Résultats (super efficace, raté, etc.)
6. **Zone des attaques** (en bas à droite)
    a. 4 boutons (grille 2x2)
    b. Affichent le nom des 4 attaques du Pokémon
    c. Désactivés pendant les animations

D. BARRES DE VIE

Fonctionnement requis :

- **Position** : Juste en dessous des infos HP
- **Fond noir** : Pour visualiser la barre qui rétrécit

Comportement :

- HP pleins (100%) → Barre VERTE
- HP moyens (50%) → Barre ORANGE

- HP faibles (25%) → Barre ROUGE
- HP à 0 → Barre à 0px (disparue)

E. SYSTÈME DE COMBAT

Déroulement d'un tour :

1. **Le joueur clique sur une attaque**
    a. Les 4 boutons sont désactivés
    b. Animation de l'attaquant
2. **Calcul des dégâts**
    a. Vérifier si l'attaque touche (précision)
    b. Si oui : calculer les dégâts
        i. Dégâts de base = puissance de l'attaque
        ii. Variation aléatoire : +0 à +10
        iii. Multiplicateur de type : x0, x0.5, x1 ou x2
    c. Si non : Afficher "L'attaque a raté !"
3. **Affichage des résultats**
    a. Message : "Pokémon utilise Attaque !"
    b. Après 1 seconde :
        i. "C'était SUPER efficace !" (x2)
        ii. "Ce n'était pas très efficace !" (x0.5)
        iii. "Cela n'a eu aucun effet !" (x0)
4. **Mise à jour des HP**
    a. Soustraire les dégâts des HP
    b. Mettre à jour l'affichage texte
    c. Animer la barre HP (rétrécissement + changement de couleur)
5. **Tour de l'ennemi**
    a. Après 2 secondes
    b. L'ennemi choisit une attaque aléatoire
    c. Même processus que le joueur
6. **Vérification de victoire**
    a. Si un Pokémon atteint 0 HP → GAME OVER
    b. Afficher le message de victoire ou défaite
    c. Proposer de recommencer
7. **Réactivation des boutons**
    a. Si les deux sont vivants, réactiver les attaques

F. ANIMATIONS (BONUS)

Points bonus pour :

- Animation d'apparition des Pokémon
- Animation lors des attaques
- Animation quand un Pokémon reçoit des dégâts
- Animation de K.O.
- Transitions fluides des barres HP
- Effets visuels selon l'efficacité

PARTIE 3 : DESIGN ET STYLE

Liberté créative :

Vous avez une **totale liberté** sur :

- Les couleurs de fond
- L'image de fond (arène, paysage, etc.)
- Le style des boutons
- Les polices de caractères
- Les effets visuels
- Les animations

Critères d'évaluation du design :

- **Cohérence** : Tous les éléments vont bien ensemble
- **Lisibilité** : Textes clairs, contraste suffisant
- **Esthétique** : Interface agréable à regarder
- **Originalité** : Votre touche personnelle

Ressources utiles :

- **Sprites Pokémon** : pokepedia.fr, pokemondb.net
- **Images de fond** : Google Images (recherche "pokemon battle background")
- **Polices** : Google Fonts ou polices Pokémon en ligne
- **Couleurs** : coolors.co pour des palettes harmonieuses

Problème 1 : La barre HP ne rétrécit pas

Erreur : Utiliser des pourcentages au lieu de pixels

```
hpBar.style.width = hpPercentage + '%'; // FAUX
```

Solution : Calculer en pixels

```
let hpWidth = Math.floor(hpPercentage * 220);
hpBar.style.width = hpWidth + 'px'; // BON
```

Problème 2 : Les HP deviennent négatifs

Erreur : Ne pas vérifier le minimum

```
receiver.hp -= damage;
```

Solution : Utiliser Math.max

```
receiver.hp -= damage;
if (receiver.hp < 0) receiver.hp = 0;
```

Problème 3 : Les boutons restent actifs pendant l'attaque

Erreur : Ne pas désactiver les boutons **Solution** : Désactiver avant l'attaque, réactiver après

```
// Désactiver
for (let i = 0; i < 4; i++) {
    document.getElementById('m' + i).disabled = true;
}
// Réactiver après 3.5 secondes
setTimeout(() => {
    for (let i = 0; i < 4; i++) {
        document.getElementById('m' + i).disabled = false;
    }
}, 3500);
```

Problème 4 : L'attaque ennemie ne se déclenche pas

Erreur : Appel immédiat

```
attack(enemyMove, pk2, pk1, 'hp1', 'Foe ');
```

Solution : Utiliser setTimeout

```
setTimeout(() => {
    attack(enemyMove, pk2, pk1, 'hp1', 'Foe ');
}, 2000); // Attendre 2 secondes
```

OBJECTIFS PÉDAGOGIQUES

À la fin de ce projet, vous serez capable de :

- ✅ Créer une application JavaScript interactive complète
- ✅ Gérer des événements utilisateur complexes
- ✅ Manipuler le DOM de manière dynamique
- ✅ Implémenter une logique de jeu avec règles
- ✅ Créer des animations CSS fluides
- ✅ Structurer un projet web propre et organisé
- ✅ Débugger et résoudre des problèmes de code

Bon courage et amusez-vous bien !